Deep Audio Zooming

Beam-width Controllable Neural Beamformer

Name: Arachchi A.D.I.D
Index: 230051L

Contents

1 Introduction **2**

2 Dataset Generation **2**
2.1 Data Sources 2
2.2 Room and Array Simulation 2
2.3 Source Placement and RIR Generation 2
2.4 Noise Addition 3
2.5 Field of View (FOV) Sampling 3
2.6 Output Format 3

3 Feature Extraction and Preprocessing **3**
3.1 Short-Time Fourier Transform 3
3.2 Log Power Spectrum (LPS) 4
3.3 Inter-channel Phase Difference (IPD) 4
3.4 Directional Features 4
3.5 FOV Pooling 4
3.6 Final Input Vector 5

4 Neural Network Architecture **5**
4.1 GRU-1 (Temporal Processing) 5
4.2 Mask Estimation 5
4.3 Subband Embedding 5
4.4 Layer Normalization + Linear Projection + Leaky ReLu 6
4.5 Per-Frequency RNN (GRU-2) 6
4.6 Filter Estimation 6
4.7 Beamforming 6
4.8 Time-Domain Reconstruction 6

5 Training and Optimization **6**
5.1 Loss Function 6
5.2 Random Field Sampling for Beamwidth Control 7
5.3 Implementation Details 7

6 Conclusion **7**

1 Introduction

The goal of the project is to develop a beamformer that can adapt to a user-defined Field of View. When we input the specific Field of view, the Neural Network gives the necessary coefficients s.t it allows the signals that are coming from the sources thatare inside the user-defined field of view while attenuating the sources that are out of the region.

2 Dataset Generation

The data set generation was done according to the steps that was described in the Deep Audio Zooming—Beamwidth Controllable Beamformer paper.

2.1 Data Sources

- **Speech**: AISHELL-2 [1], a large-scale Mandarin speech corpus recorded at 16 kHz. It contains thousands of utterances from multiple speakers, split into train/dev/test sets.
- **Noise**: DEMAND [2], a database of 16 environmental noises (e.g., kitchen, traffic, office) also at 16 kHz. Various noise types are used to add background interference.

2.2 Room and Array Simulation

We created random rooms that are in the regions of the following ranges for x,y,z directions:

$$L_x \in [4, 8]\,\mathrm{m}, \quad L_y \in [4, 8]\,\mathrm{m}, \quad L_z \in [2.5, 4]\,\mathrm{m}.$$

An 8-channel circular microphone array of radius 5 cm is placed at a random position within the room, at least 0.5 m away from the walls, at a height between 1.0 m and 1.5 m. The array is assumed to look along the positive x-axis (azimuth 0° corresponds to the front).

2.3 Source Placement and RIR Generation

- We randomly changed the source speaker number form 1-5 and their positions are randomly sampled inside the room, with a minimum distance of 0.5 m from the array centre.
- Room impulse responses (RIRs) are computed using the image-source method implemented in `pyroomacoustics` [3] with a maximum reflection order corresponding to a reverberation time T_{60} randomly chosen from $[0.3, 1.3]$ s.
- Each dry speech signal is convolved with its respective RIR, and the contributions are summed to obtain the clean multi-channel mixture.

2.4 Noise Addition

- Popular environmental noise dataset DEMAND was used to add the noise to the data set.
- Changed the signal-to-noise ratio (SNR) uniformly sampled from 10 to 40 dB and according to its value scaled the noise power.
- The noise source is placed at a random location in the room, convolved with its own RIR.

2.5 Field of View (FOV) Sampling

- At the beginning, we assumed that the sources are in a 2D plane. So the user FOV is a 2D plane.
- Target FOV is randomly chosen: an azimuth range with width between 30° and 180° and a random centre. (Elevation is kept fixed at 0° for simplicity, but the method can be extended to 3D.)
- Speakers whose positions fall inside this angular range are considered the target sources; the remaining speakers (if any) are interferers. The target signal is defined as the *anechoic* sum of the dry speech signals of those target speakers.
- If no speaker lies inside the initially sampled FOV, the FOV is resampled up to 10 times; after that, the first speaker is arbitrarily taken as target.

2.6 Output Format

For every generated utterance, the following files are saved:

- `*_mix.wav`: 8-channel mixture (with noise), 16 kHz.
- `*_target.wav`: mono target signal (anechoic sum of sources inside FOV).
- `*_meta.json`: metadata containing source positions, room dimensions, T_{60}, SNR, FOV boundaries, and indices of sources inside FOV.

The dataset is split into train, validation, and test subsets, with sizes matching the paper (e.g., 95k / 2.5k / 100) when scaled appropriately.

3 Feature Extraction and Preprocessing

Before feeding data into the neural network, raw waveforms are transformed into a compact time-frequency representation augmented with directional information.

3.1 Short-Time Fourier Transform

Each channel c is processed with a Hann window (FFT size $N_{\text{FFT}} = 512$, hop length $H = 256$) to obtain complex spectrograms:

$$Y_c(t, f) \in \mathbb{C}, \quad c = 1, \ldots, 8,$$

where t denotes the time frame index and f the frequency bin.

3.2 Log Power Spectrum (LPS)

From a reference channel (typically the first), we compute the log power spectrum:

$$L(t, f) = \log\left(|Y_1(t, f)|^2 + \epsilon\right),$$

with ϵ a small constant to avoid numerical issues.

3.3 Inter-channel Phase Difference (IPD)

The IPD between the reference microphone and each other microphone c is:

$$\Gamma_c(t, f) = \frac{Y_c(t, f)}{|Y_c(t, f)|} \cdot \frac{\overline{Y_1(t, f)}}{|Y_1(t, f)|}.$$

- $Y_c(t, f)$ is the STFT of the channel c
- $Y_1(t, f)$ is the STFT of the reference channel.

These complex numbers have unit magnitude and encode the phase difference.

$$\Gamma_c(t, f) = e^{j(\phi_c(t,f) - \phi_1(t,f)}$$

3.4 Directional Features

We compare the observed IPD with the theoretical IPD expected for a far-field source from direction θ. The theoretical IPD for microphone c is:

$$\Gamma_\theta^{c,\text{theory}}(f) = e^{-j2\pi f \tau_c(\theta)},$$

where $\tau_c(\theta)$ is the time difference of arrival between microphone c and the reference, computed from the array geometry and the speed of sound $c_{\text{sound}} = 343\,\text{m/s}$.

$$\tau_c(\theta) = \Delta_m.cos(\theta)/c$$

Where Δ_m is the distance between reference channel and the microphone m. and θ is the steering direction.

For each candidate direction θ (e.g., every 1° in azimuth), we compute a similarity score:

$$d_\theta(t, f) = \text{Re}\left(\sum_{c=2}^{8} \Gamma_c(t, f) \cdot \overline{\Gamma_\theta^{c,\text{theory}}(f)}\right).$$

This score ranges from -1 (opposite phase) to $+1$ (perfect alignment). Values near $+1$ indicate that the T-F bin is likely dominated by a source from direction θ.

3.5 FOV Pooling

Given a user-defined FOV azimuth range $[\Theta_l, \Theta_h]$, we aggregate the directional information into two scalar features per T-F bin:

$$D_{\text{in}}(t, f) = \max_{\theta \in [\Theta_l, \Theta_h]} d_\theta(t, f), \qquad D_{\text{out}}(t, f) = \max_{\theta \notin [\Theta_l, \Theta_h]} d_\theta(t, f).$$

If the FOV contains no candidates, D_{in} is set to zero. These features tell the network how strongly each bin is associated with sources inside versus outside the desired region.

3.6 Final Input Vector

For every time-frequency bin, the three features are concatenated:

$$\mathbf{f}(t,f) = \big[L(t,f),\ D_{\text{in}}(t,f),\ D_{\text{out}}(t,f)\big]^\top.$$

This 3-dimensional vector serves as the input to the neural model. All features are computed offline or on-the-fly during training; the process is differentiable and can be integrated into the training pipeline.

4 Neural Network Architecture

The model is a subband neural beamformer designed for low computational cost (approx. 860k parameters, 184MMACs/s). It operates independently on each frequency band, sharing weights across bands except where noted.

4.1 GRU-1 (Temporal Processing)

For each frequency band f, the sequence $\mathbf{f}(t,f)$ over time is fed into a unidirectional Gated Recurrent Unit (GRU) with hidden size $H_{\text{gru}} = 128$. This layer captures temporal dynamics of speech and interference:

$$\mathbf{h}_1(t,f) = \text{GRU}_1\big(\mathbf{f}(t,f)\big), \quad \mathbf{h}_1 \in \mathbb{R}^{H_{\text{gru}}}.$$

4.2 Mask Estimation

- In mask estimation, two are independent linear layers are used.
- Output is 2 Masks, where each mask is combined with real and imaginary parts that are coming out of 2 linear layers.

$$M_{\text{in}}(t,f),\ M_{\text{out}}(t,f) \in \mathbb{C},$$

- $M_{\text{in}}(t,f),\ M_{\text{out}}(t,f)$ defines the source estimations within and outside of the FOV.

4.3 Subband Embedding

Using the reference microphone spectrum $Y_1(t,f)$, we compute initial estimates:

$$S_{\text{in,c}}(t,f) = M_{\text{in}}(t,f) \cdot Y_c(t,f), \qquad S_{\text{out,c}}(t,f) = M_{\text{out}}(t,f) \cdot Y_c(t,f).$$

The real and imaginary parts of Y_1, S_{in}, and S_{out} are concatenated to form a 6-dimensional embedding:

$$\mathbf{e}(t,f) = \big[\text{Re}(S_{\text{in},1}), \text{Im}(S_{\text{in},1}), ..., \text{Re}(S_{\text{in},8}), \text{Im}(S_{\text{in},8}), \text{Re}(S_{\text{out},1}), \text{Im}(S_{\text{out},1}), ..., \text{Re}(S_{\text{in},8}), \text{Im}(S_{\text{in},8})\big].$$

This embedding is passed through layer normalization and a linear projection, reducing it to $H_{\text{gru}} = 128$ dimensions.

4.4 Layer Normalization + Linear Projection + Leaky ReLu

Layer normalization computation :-

$$e' = \frac{e - \mu}{\sqrt{\sigma^2 + \epsilon}}.\gamma + \beta$$

- μ - mean, σ - standard deviation
- γ, β - learnable params

4.5 Per-Frequency RNN (GRU-2)

A second GRU (implemented as a cell applied iteratively over time) processes each frequency band independently, refining the representations:

$$\mathbf{h}_2(t, f) = \mathrm{GRU}_2\big(\mathbf{e}(t, f)\big).$$

4.6 Filter Estimation

A linear layer transforms $\mathbf{h}_2(t, f)$ into complex beamforming weights for every microphone:

$$W_c(t, f) \in \mathbb{C}, \quad c = 1, \ldots, 8,$$

again represented by two outputs (real, imag) per microphone.

4.7 Beamforming

The enhanced spectrogram is obtained by a weighted sum across all microphone channels:

$$\hat{S}(t, f) = \sum_{c=1}^{8} W_c^*(t, f) \cdot Y_c(t, f),$$

where * denotes complex conjugation. This operation adaptively combines channels to emphasise sounds from inside the FOV.

4.8 Time-Domain Reconstruction

Finally, the enhanced spectrogram $\hat{S}(t, f)$ is transformed back to the time domain via inverse STFT (using the same window and hop) to yield the output waveform $\hat{s}(n)$.

5 Training and Optimization

5.1 Loss Function

The model is trained end-to-end with a combination of a time-domain loss and a frequency-domain loss:

- **Negative SI-SDR** (Scale-Invariant Signal-to-Distortion Ratio):

$$\mathcal{L}_{\text{SI-SDR}} = -20 \log_{10} \frac{\|\alpha \cdot s_{\text{target}}\|}{\|\hat{s} - \alpha \cdot s_{\text{target}}\|}, \quad \alpha = \frac{\hat{s}^\top s_{\text{target}}}{\|s_{\text{target}}\|^2}.$$

- **Spectral L1 loss** on the magnitude spectra:

$$\mathcal{L}_{\text{spec}} = \big\| |\hat{S}(t,f)| - |S_{\text{target}}(t,f)| \big\|_1.$$

The total loss is:

$$\mathcal{L} = \mathcal{L}_{\text{SI-SDR}} + \lambda \cdot \mathcal{L}_{\text{spec}},$$

with $\lambda = 0.1$ weighting the spectral term.

5.2 Random Field Sampling for Beamwidth Control

To make the model beamwidth-controllable, during training we randomly sample a new FOV for each utterance (width uniformly between 30° and 180°, centre uniformly random). The features D_{in} and D_{out} are recomputed accordingly, and the target signal is formed as the anechoic sum of speakers falling inside that newly sampled FOV. This forces the network to learn to adapt its behaviour to different angular regions.

5.3 Implementation Details

Training is performed with the Adam optimizer (learning rate 10^{-3}) and a batch size of 8. Gradients are clipped to a maximum norm of 5. The model is implemented in PyTorch and trained on a single NVIDIA GPU for approximately 100 epochs. Data augmentation includes random gain perturbations and occasional dropout of speakers to improve robustness.

6 Conclusion

We have presented a complete pipeline for deep audio zooming: from realistic dataset generation using AISHELL-2 and DEMAND, through physics-inspired feature extraction (LPS, IPD, directional scores, FOV pooling), to a low-complexity subband neural beamformer. The model, trained with random field sampling, learns to enhance sources inside an arbitrary field of view while suppressing interferers, making it suitable for real-time applications such as hearing aids, smart speakers, and video conferencing.

References

[1] J. Du, X. Na, X. Liu, and H. Bu, *AISHELL-2: Transforming Mandarin ASR Research into Industrial Scale*, 2018.

[2] J. Thiemann, N. Ito, and E. Vincent, *The Diverse Environments Multi-channel Acoustic Noise Database (DEMAND)*, 2013.

[3] R. Scheibler, E. Bezzam, and I. Dokmanić, *Pyroomacoustics: A Python package for audio room simulation and array processing algorithms*, 2018.